Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ДОМАШНЕЕ ЗАДАНИЕ №5

Реализация межсервисного взаимодействия посредством очередей сообщений

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ВВЕДЕНИЕ

Данный отчёт посвящён анализу и реализации межсервисного взаимодействия в системе Callmentor через брокер сообщений RabbitMQ. Использование очередей сообщений является ключевым архитектурным решением для обеспечения асинхронной обработки задач машинного обучения, что позволяет системе эффективно справляться с высокой нагрузкой без потери сообщений.

2. ТЕХНОЛОГИЯ: RABBITMQ

RabbitMQ — это брокер сообщений с открытым исходным кодом, реализующий протокол AMQP 0-9-1. В проекте используется образ rabbitmq:3.13-management. Ключевые характеристики:

- Надёжная доставка сообщений (durable queues, persistent messages).
- Маршрутизация через Exchange (тип DIRECT с routing keys).
- Управление потоком (prefetch, QoS).
- Web Management UI (порт 15672) для мониторинга очередей.
- Healthcheck через rabbitmq-diagnostics.

3. АРХИТЕКТУРА ОЧЕРЕДЕЙ

3.1. Exchange и маршрутизация

Все сервисы используют единый Exchange с именем "exchange" типа DIRECT и флагами durable=True, auto_delete=False. Каждое сообщение публикуется с routing key, равным имени целевой очереди. Такой подход обеспечивает прямую адресацию сообщений к конкретным обработчикам.

3.2. Гарантии доставки

Все очереди создаются с параметрами durable=True (персистентность при рестарте RabbitMQ) и auto_delete=False (очередь не удаляется при отключении потребителей). Сообщения публикуются с флагами persist=True и mandatory=True, что гарантирует их сохранение на диск и возврат ошибки при невозможности маршрутизации.

3.3. Parking-очереди

Для каждой критической очереди существует соответствующая parking-очередь (dead letter queue). Parking-очереди инициализируются при старте сервисов через

ensure_parking_queues(). При невосстановимой ошибке обработчик не ACK-ит сообщение, а отправляет его в parking-очередь для последующего анализа:

- CALLBACK_ERROR_QUEUE — ошибки отправки колбэка в Gateway.
- ASR_MANUAL_REVIEW_QUEUE — звонки, требующие ручной проверки ASR.
- LLM_TASK_ERROR_QUEUE — ошибки LLM-анализа.
- LLM_POSTPROCESSING_ERROR_QUEUE — ошибки постобработки.

4. РЕАЛИЗАЦИЯ ПРОДЮСЕРОВ

4.1. ML-service как продюсер

ML-service использует FastStream RabbitBroker для публикации сообщений. Объект broker инициализируется из конфигурации RABBIT_URL. Публикация выполняется методом await broker.publish() с указанием очереди, exchange и атрибутов персистентности. Код маршрутизации в ml_core.py:

При обнаружении стерео-аудио с различными каналами → публикация в ASR_TRANSCRIPTION_QUEUE со структурой {task_id, audio_path, prompts, metadata, model_type}. При обнаружении моно → ASR_DIARIZATION_QUEUE с аналогичной структурой. После конвертации задвоенного стерео в моно → также ASR_DIARIZATION_QUEUE.

4.2. ml-asr-worker как продюсер

После завершения транскрибации ASR-worker публикует результат в LLM_TASK_QUEUE. Сообщение содержит: task_id (UUID задачи), transcription (текст транскрипции), prompts (набор промтов компании), metadata (метаданные из CRM), model_type (выбранный LLM-провайдер).

4.3. ml-llm-worker как продюсер

LLM-worker публикует результаты анализа в CALLBACK_QUEUE. Сообщение содержит структурированный ответ с результатами всех промтов (arterfacts), суммаризацией (summary) и рекомендациями (recommendations). При ошибке публикует в LLM_TASK_ERROR_QUEUE или LLM_POSTPROCESSING_ERROR_QUEUE.

5. РЕАЛИЗАЦИЯ КОНСЬЮМЕРОВ

5.1. ml-asr-worker как консьюмер

ASR-worker использует декораторы FastStream для объявления обработчиков очередей. Обработчики реализованы как async-функции с параметром типа Pydantic-модели. FastStream автоматически десериализует JSON-сообщения в модели:

- _transcription_subscriber: @broker.subscriber(ASR_TRANSCRIPTION_QUEUE) — прямая транскрибация стерео.
- _diarization_subscriber: @broker.subscriber(ASR_DIARIZATION_QUEUE) — диаризация + транскрибация моно.
- _manual_review_subscriber: @broker.subscriber(ASR_MANUAL_REVIEW_QUEUE) — обработчик parking-очереди.

5.2. ml-llm-worker как консьюмер

LLM-worker аналогично объявляет обработчики для LLM-очередей:

- _llm_audio_subscriber: основной обработчик LLM-задач.
- _kpi_subscriber: обработчик KPI-отчётов.
- _objections_subscriber: обработчик кластеризации возражений.
- _postprocess_subscriber: постобработка результатов.
- _llm_task_error_subscriber: обработчик parking LLM-ошибок.
- _postprocess_error_subscriber: обработчик parking ошибок постобработки.

5.3. ml-service как консьюмер

ML-service обрабатывает CALLBACK_QUEUE и CALLBACK_ERROR_QUEUE. При получении успешного колбэка (CALLBACK_QUEUE) отправляет HTTP-запрос на Gateway (/ml-callback) с результатами анализа. При ошибке (CALLBACK_ERROR_QUEUE) логирует и сохраняет информацию для последующего анализа.

6. МОДЕЛИ СООБЩЕНИЙ

Таблица 1 — Модели сообщений в очередях

Очередь	Ключевые поля сообщения
ASR_TRANSCRIPTION_QUEUE	task_id: UUID, audio_path: str, prompts: dict, metadata: dict, model_type: str
ASR_DIARIZATION_QUEUE	task_id: UUID, audio_path: str, prompts: dict, metadata: dict, model_type: str
LLM_TASK_QUEUE	task_id: UUID, transcription: str, prompts: dict, metadata: dict, model_type: str

CALLBACK_QUEUE	task_id: UUID, artefacts: list[dict], summary: str, recommendations: str
KPI_REPORT_QUEUE	task_id: str, manager_name: str, call_summaries: list[str], prompts_counts: dict, total_calls: int, model_type: str

7. RETRY-МЕХАНИЗМ

Для обработки временных сбоев (например, недоступность LLM API) в ml-llm-worker реализован класс RetryableLLMError. При возникновении такой ошибки сообщение не ACK-ится и автоматически возвращается в очередь для повторной обработки. Постоянные ошибки направляются в parking-очереди.

8. ВЫВОДЫ

Реализована надёжная система межсервисного взаимодействия через RabbitMQ. Архитектура обеспечивает: гарантированную доставку сообщений через durable-очереди и persistent-сообщения, обработку ошибок через parking-очереди, автоматический retry при временных сбоях, трассировку задач по полю task_id, горизонтальное масштабирование воркеров за счёт consumer groups.